

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Embedded Systems (CO3053)

Assignment 2 Report

Washing Machine Control Unit

State Design and Software Implementation

Advisor: Dr. Phạm Hoàng Anh

Students: Nguyễn Viết Tấn Vương 2353345

Nguyễn Lê Cao Tường 2353300

Vũ Tiến Vinh 2353334

HO CHI MINH CITY, OCTOBER 2026

Contents

1	Problem and Requirements	1
1.1	Required Behavior	1
1.2	Choices for Details Not Given in the Brief	1
1.3	Scope of the Submitted Evidence	1
2	Design	2
2.1	Modules and Data Flow	2
2.2	State Machine	2
2.3	Outputs in Each State	2
2.4	Transition Rules	3
2.5	Timer and Pause Behavior	3
2.6	Work in Each Loop	3
3	Implementation	4
3.1	Proposed Board Connection	4
3.2	Code Structure	5
3.3	Important Code	5
3.4	Running the Demonstration	5
4	Verification and Results	6
4.1	Software Tests	6
4.2	Simulation Evidence	7
4.3	What Still Needs a Physical Check	7
5	Team Work and Review Plan	8
5.1	Proposed Work Split	8
5.2	Schedule with Two Review Days	8

Project files. The source package contains the report, controller code, an offline browser simulator, diagrams and test results. The report presents a software implementation and a proposed board connection.

1 Problem and Requirements

The project controls a washing machine with coins, three buttons and two LEDs. A customer pays at least 50 cents and presses RUN to start a 30 minute cycle. PAUSE stops washing while the timer continues. RUN resumes the same cycle. The controller returns to standby after the time ends or after two STOP presses. These rules come from the assignment sheet [1].

The implementation covers the control unit. Washing is represented by a digital enable signal. Water valves, drum speed and wash stages are outside the given task.

1.1 Required Behavior

ID	Requirement
R1	Use STOP, RUN and PAUSE as the three machine controls.
R2	Accept only 10, 20 and 50 cent coins. Add their values to the credit.
R3	Keep RLED on in standby. Make RLED blink when an error occurs.
R4	Turn BLED on when credit reaches at least 50 cents.
R5	Start only after enough credit and a RUN press. Clear all credit, including surplus.
R6	Start a 30 minute timer on the first RUN. Make BLED blink while washing.
R7	On PAUSE, stop washing but keep the timer counting. RUN resumes without restarting it.
R8	End at 30 minutes, including paused time, or after two separate STOP presses.

1.2 Choices for Details Not Given in the Brief

Detail	Choice used in this project
Ready indication	Ready is still available before a cycle, so RLED stays on and BLED is on.
Paused indication	RLED is off, BLED stays on and the wash output is off.
Two STOP presses	Count two separate press events in one active cycle. No time window is imposed. PAUSE and RUN keep the count.
Active cycle coins	Ignore coins in Running, Paused and Error. The coin input is a bench emulator of a validated acceptor.
Error and recovery	A test fault signal enters Error. Washing stops and credit clears. Clear the fault and reset the board to recover.
Blink and buttons	Use a 1 second blink period and 30 ms button debounce. A held button gives one event.

1.3 Scope of the Submitted Evidence

The package includes working controller logic, a browser simulation and automated software tests. It also includes a board sketch and a wiring plan. Physical board operation has not been measured. The supplied brief does not specify a board or a required photograph; any separate lab instruction must be checked before submission.

2 Design

2.1 Modules and Data Flow

Input handling turns stable presses and accepted coins into events. The state controller stores credit, the cycle start time and the STOP count. The timer checks elapsed time. Output logic sets both LEDs and the wash enable signal.

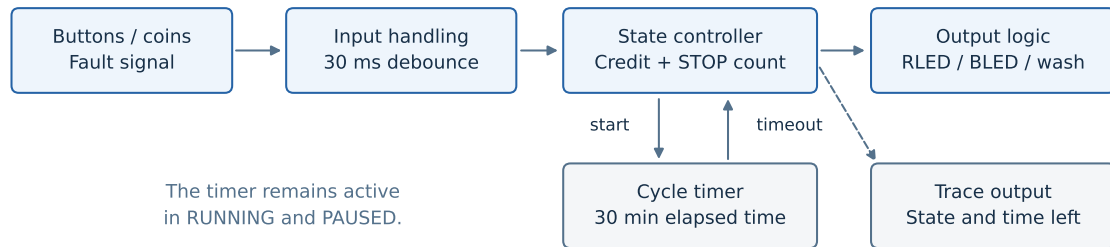


Figure 1: Main modules and their data flow.

2.2 State Machine

Use five states: Standby, Ready, Running, Paused and Error. Figure 2 shows the main transitions. Coin collection below 50 cents stays in Standby.

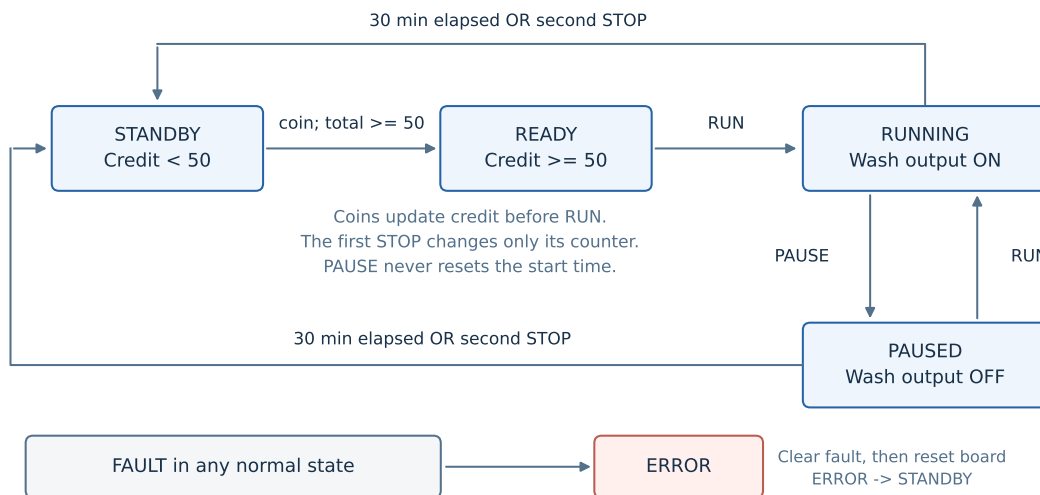


Figure 2: Main state transitions. A fault can enter Error from any normal state.

2.3 Outputs in Each State

State	RLED	BLED	Wash output	Timer
Standby	On	Off	Off	Not active
Ready	On	On	Off	Not active
Running	Off	Blink	On	Counting
Paused	Off	On	Off	Counting
Error	Blink	Off	Off	Cycle cancelled

Blink means 500 ms on and 500 ms off. The ready and paused combinations follow the explicit choices in Section 1.2.

2.4 Transition Rules

Current state	Event or condition	Result
Standby	Valid coin; credit below 50	Add credit; stay in Standby.
Standby	Valid coin; credit reaches 50	Add credit; enter Ready.
Ready	Valid coin	Add credit; stay in Ready.
Ready	RUN	Clear credit, save start time and enter Running.
Running	PAUSE	Turn washing off; enter Paused. Keep start time.
Paused	RUN	Turn washing on; enter Running. Keep start time.
Running or Paused	First STOP	Set STOP count to 1. Keep the current state.
Running or Paused	Second STOP	Clear cycle data; enter Standby.
Running or Paused	Elapsed time reaches 30 min	Clear cycle data; enter Standby.
Any normal state	Fault signal	Turn washing off, clear credit and enter Error.
Error	Board reset after fault is cleared	Return to Standby with zero credit.

Unsupported coins do not add credit. Other unused controls leave the state unchanged. RUN in Running does not restart the timer.

2.5 Timer and Pause Behavior

The timer uses elapsed time from the first RUN:

$$T_{\text{cycle}} = 30 \times 60 \times 1000 = 1,800,000 \text{ ms}, \quad T_{\text{left}} = \max(0, T_{\text{cycle}} - (t - t_{\text{start}})).$$

The start time changes only when a paid cycle starts. For example, pause at minute 8 and resume at minute 16. There are 14 minutes left after resuming, and the cycle still ends at minute 30.

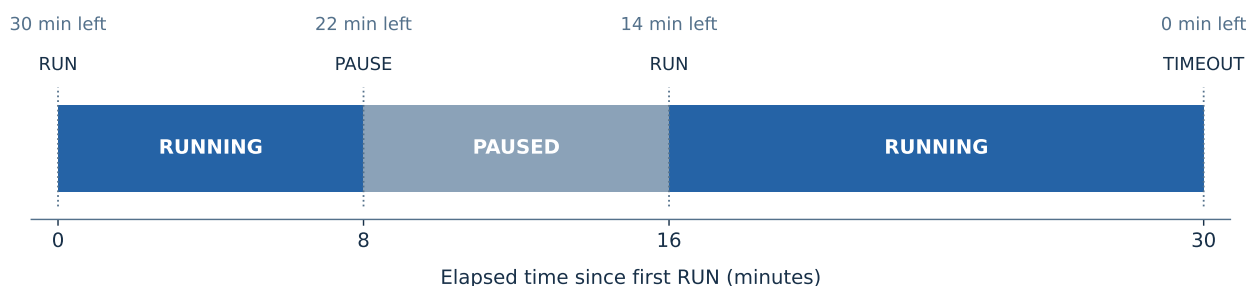


Figure 3: Example cycle with eight minutes spent in Paused.

2.6 Work in Each Loop

1. Read the inputs and create debounced press events.
2. Check the fault signal. A fault has the highest priority.
3. Check the deadline in both Running and Paused.
4. Collect valid coins before a cycle, or process STOP, PAUSE and RUN during a cycle.
5. Update the LEDs and wash output from the current state.
6. Write a short trace when the state, credit or STOP count changes.

In an active cycle, STOP is processed before the other buttons. PAUSE takes priority over RUN when the machine is Running. An expired cycle ignores input events from that same loop. The loop uses no blocking wait.

3 Implementation

3.1 Proposed Board Connection

The example target is Arduino UNO R3. Its digital inputs and outputs are enough for this small control unit [2]. Three extra switches emulate coin signals; they are test inputs, not extra machine controls. An optional fault switch provides the error signal.

Signal	Pin	Connection
STOP, RUN, PAUSE	D2, D3, D4	Each switch connects its input to GND.
10, 20, 50 cent test inputs	D5, D6, D7	Each switch models one valid coin insertion.
RLED	D8	D8 to 330 Ω resistor, LED anode, then cathode to GND.
BLED	D9	D9 to 330 Ω resistor, LED anode, then cathode to GND.
Wash enable	D10	Digital enable for a separate driver; high only in Running.
Fault test input	A0	Switch to GND; LOW enters Error.
Power and reference	USB, GND	USB supplies the board; all test inputs share GND.

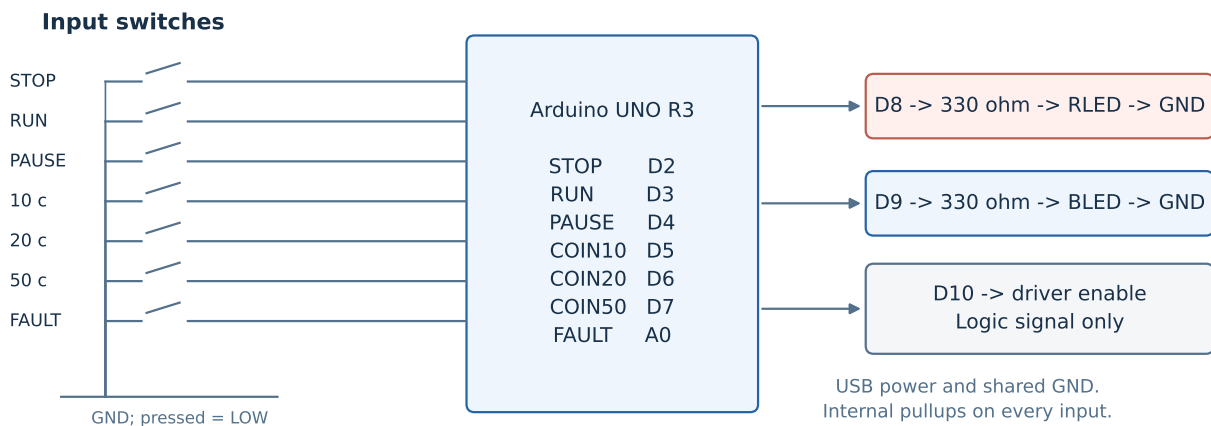


Figure 4: Proposed signal connections for the bench circuit.

All switch inputs use INPUT_PULLUP. An open switch reads HIGH and a pressed switch reads LOW [3]. Each button must stay stable for 30 ms before a press is counted. Release is also debounced, so a held STOP cannot be mistaken for two presses.

The bench parts are one UNO R3, two LEDs, two 330 Ω resistors, six push buttons, one optional fault switch, a breadboard, jumper wires and a USB cable. Coins can also be entered through Serial Monitor, so the three coin switches are optional for a simple demonstration.

D10 represents the control decision. A motor requires a separate driver and power supply. No motor or mains circuit is needed to show the required state logic.

3.2 Code Structure

The controller is written in C++. It does not call board functions, so the same code can be tested on a computer. The sketch only reads inputs, calls the controller and writes outputs. The browser model follows the same state rules.

File	Purpose
controller.h	States, credit, timer, STOP count, outputs and debounce.
washing_controller.ino	Pin setup, physical input reading and Serial test commands.
simulator/index.html	Offline page with coins, buttons, LEDs, timer and event log.
simulator/controller.js	Browser version of the state controller.
tests/test_controller.py	Compiles the C++ tests and checks expected results.
tests/test_firmware.cpp	Runs the actual sketch with mocked board inputs and outputs.

3.3 Important Code

The deadline is checked before control buttons. Both active states use the same start time. These lines are taken from the controller:

```
if (active() && uint32_t(now - started_) >= CycleMs) {
    finish(now);
    return;
}
if (in.stop) {
    ++stop_count_;
    if (stop_count_ >= 2) finish(now);
    return;
}
if (in.pause && state_ == State::Running) {
    enter(State::Paused, now);
} else if (in.run && state_ == State::Paused) {
    // Keep started_: a pause never gives extra time.
    enter(State::Running, now);
}
```

The sketch reads `millis()` and uses unsigned subtraction for elapsed time. It updates the timer and LEDs without `delay()`, following the idea of Arduino's Blink Without Delay example [4]. The tests also cover clock rollover and button bounce.

3.4 Running the Demonstration

Open `simulator/index.html` in a browser. Insert coins, press RUN, advance eight minutes and press PAUSE. Advance another eight minutes and press RUN. The display should show 14:00 remaining. Advance to minute 30 to verify timeout. Test two STOP presses in a new cycle, then inject a fault.

The default clock is manual. Real time runs at 1x; demo time runs at 60x. Both use the same 30 minute virtual cycle. The browser can export the event log.

For the board sketch, open `washing_controller.ino` with its adjacent `controller.h`. Serial commands are 1, 2, 5 for coins and r, p, s for controls at 115200 baud. Verify and upload to the chosen board before using it in a physical demonstration.

4 Verification and Results

4.1 Software Tests

Tests run the C++ controller with a virtual millisecond clock. Each scenario checks expected states, credit, remaining time or outputs. No real 30 minute wait is needed: the tests supply time values directly. These results verify logic, not the accuracy of a physical board clock.

Check	Measured result
Required behavior and edge cases	25 scenarios passed; 145 expected fields checked.
Button handling	13 checks passed for bounce, held presses, release and rollover.
Sketch adapter on computer	15 checks passed with mocked Arduino inputs and outputs.
Browser model against C++	All 5,001 steps matched in a seeded exploratory trace.
UNO R3 target build	Compiled with Arduino AVR Boards 1.8.6; 3,838 bytes of flash and 302 bytes of static RAM.
Browser controls	Coins, buttons, clock and fault controls passed; 10 captures saved.

Test	Behavior checked	Result
T01	Standby LEDs and disabled wash output.	Pass
T02	RUN ignored with only 30 cents.	Pass
T03	20 + 20 + 10 cents enters Ready.	Pass
T04	One 50 cent coin enters Ready.	Pass
T05	70 cents becomes zero when washing starts.	Pass
T06	A 25 cent input is rejected.	Pass
T07	Coins ignored in Running and Paused.	Pass
T08	BLED changes every 500 ms while Running.	Pass
T09	Time decreases while Paused.	Pass
T10	Resume keeps the original deadline.	Pass
T11	Running ends exactly at the 30 minute boundary.	Pass
T12	Paused also ends at the 30 minute boundary.	Pass
T13	First STOP keeps Running; second STOP ends it.	Pass
T14	Two STOP presses also work in Paused.	Pass
T15	STOP count survives pause and resume.	Pass
T16	A new cycle clears the old STOP count.	Pass
T17	Fault disables washing and makes RLED blink.	Pass
T18	Error stays latched until reset.	Pass
T19	Repeated RUN cannot restart the timer.	Pass
T20	Cycle timing works across unsigned clock rollover.	Pass
T21	Fault takes priority over control commands.	Pass
T22	Timeout takes priority over resume.	Pass
T23	PAUSE takes priority over RUN while Running.	Pass
T24	STOP takes priority over PAUSE.	Pass
T25	Unused controls do not change Standby.	Pass

Run `python3 tests/test_controller.py` from the project folder. It needs Python 3, a C++ compiler named `g++`, and Node.js. The `evidence/` folder contains the console output, result table and traces.

4.2 Simulation Evidence

These images are captured from the local software simulator. They show the control behavior, not a photograph of an assembled machine.

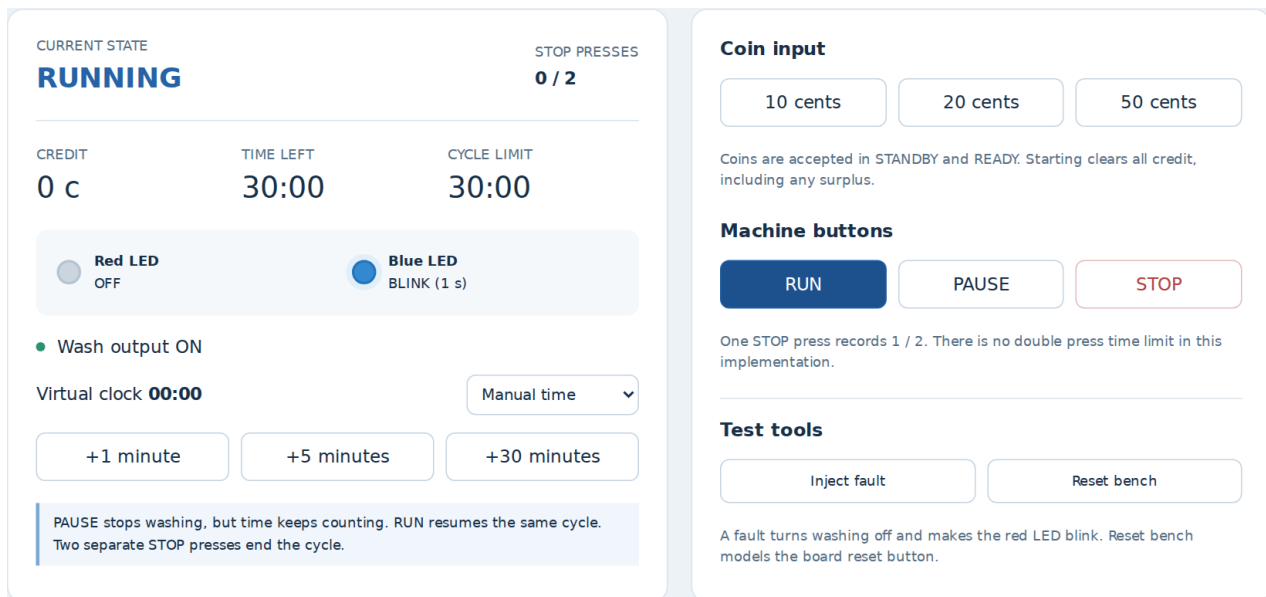


Figure 5: Running after payment of 70 cents. Credit is cleared and the cycle starts at 30:00.

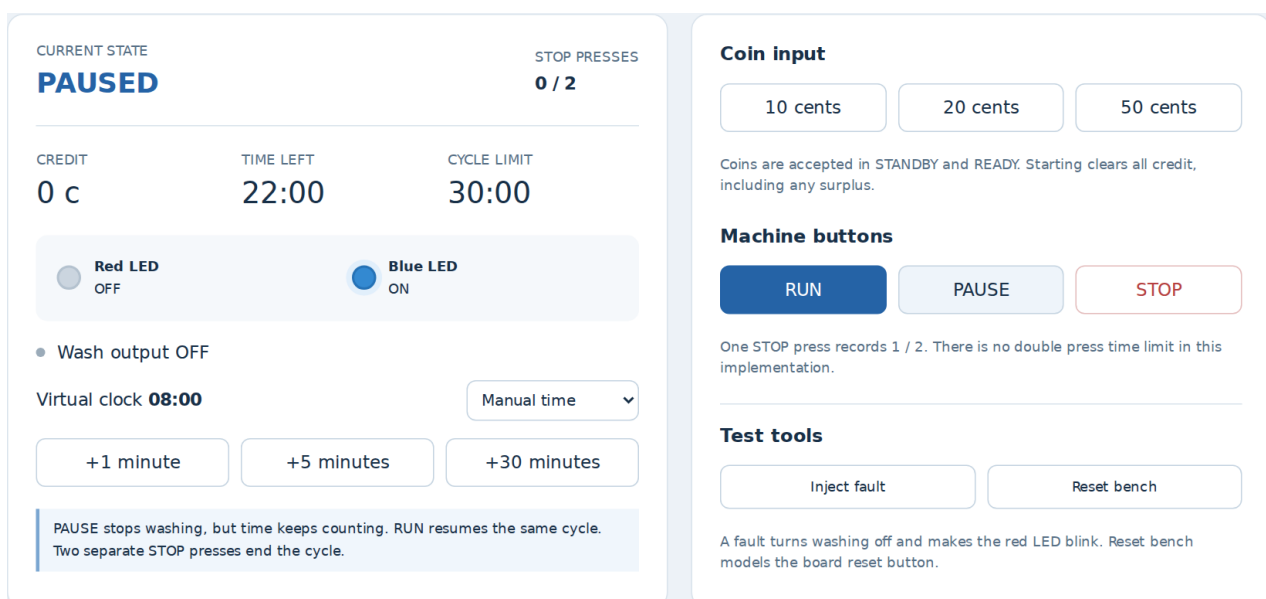


Figure 6: Paused at minute 8. Washing is off and 22:00 remains on the same timer.

The stored demonstration continues to minute 16 and resumes with 14:00 left. At minute 30, the state returns to Standby. Separate captures show the first STOP press, force stop after the second press, and the Error state. The saved event log is `evidence/browser_cycle_log.json`.

4.3 What Still Needs a Physical Check

The physical circuit has not been assembled or tested in this work. If a board demonstration is required, upload the compiled sketch and check the LEDs, controls and timeout. Record the board and Serial Monitor together. The software tests do not measure real switch bounce, wiring quality, coin acceptor signals or a motor driver.

5 Team Work and Review Plan

5.1 Proposed Work Split

The following split gives each member a clear output. Each person should explain their part and review one other part before submission.

Member	Responsibility and output
Nguyễn Viết Tấn Vương 2353345	Requirements, explicit assumptions, module diagram, state diagram and transition table. Review the code against the rules.
Nguyễn Lê Cao Tướng 2353300	C++ controller, button debounce, board sketch and pin mapping. Review the simulator's timer, outputs and STOP behavior.
Vũ Tiến Vinh 2353334	Browser demo, test evidence, report assembly and PDF checks. Review the state table and all submission files.

5.2 Schedule with Two Review Days

Use a seven day plan only if seven days are available; the brief supplied here does not give a deadline. Reserve the last two days for review.

Period	Work and completion check
Days 1 and 2	Agree on requirements and state rules. Finish diagrams and a first working controller.
Days 3 and 4	Complete the sketch and simulator. Run coin, pause, timeout, STOP and error tests.
Day 5	Merge all files. Finish the report and freeze new features.
Day 6	Review each other's parts. Repeat failed tests and check any required board demonstration.
Day 7	Check names, IDs, figures, contents, source package and submission rules. Submit the final files.

The controller implements the coin threshold, timed cycle, pause and resume, two press stop and error indication. The reported results are software tests. A physical demonstration, if required separately, remains a group check.

References

- [1] CO3053, *Assignment 2: Washing Machine Control Unit*, provided assignment sheet, Lecture Notes slide 33.
- [2] Arduino, *UNO R3*. <https://docs.arduino.cc/hardware/uno-rev3/>
- [3] Arduino, *INPUT, INPUT_PULLUP, OUTPUT*. <https://docs.arduino.cc/language-reference/en/variables/constants/inputOutputPullup/>
- [4] Arduino, *Blink Without Delay*. <https://docs.arduino.cc/built-in-examples/digital/BlinkWithoutDelay/>